Name-> Suraj Kumar

Roll NO. -> M25CSA030

CSL 7640: NATURAL LANGUAGE UNDERSTANDING

Assignment-2 REPORT

Q1) Task 1.)

Output Of Code

```
[nltk_data] Downloading package punkt to /home/suraj/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /home/suraj/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
Total Documents: 3
Total Tokens: 28089
Vocabulary Size: 5868
Tokens in Document 1: 968
Tokens in Document 2: 22507
Tokens in Document 3: 4614

Top 20 Frequent Words:
learning : 373
data : 347
computer : 301
course : 297
algorithms : 232
systems : 204
design : 204
science : 180
networks : 176
tech : 169
engineering : 161
elective : 155
computing : 153
level : 152
students : 151
edition : 149
analysis : 139
applications : 138
research : 133
department : 130
```

Word Cloud

application
algorithm
system
design
department computer
content
learning outcome
network
model
data
technique
computer science
science engineering
sankar banerjee
problem
objective
none
dip sankar
outcomes student
elective course
offered tech
machine learning
course title
level elective
graph
using
optimization
data structure
reference book
antirequisite none
elective prerequisite
expected ability
students expected
type elective
protocol
computing
management
architecture
basic
concept
ph type
tech ph
various
big data
social network
research papers
deep learning
blockchain
security
understanding
principle
text book
analysis
gaurav harit
tech tech
learning material
structure offered
artificial intelligence
press edition
type compulsory
cryptography
advanced
program
video
knowledge
environment
international conference
dynamic
graphic
number csl
self learning
santanu chaudhury
programming
computer network
research
operating system
computer vision
university press
communication
information
objectives instructor
memory
textbook
method
structures algorithm
kernel
course csl
software
based
engineering structure
computer graphics
kishore kothapalli
search
work
challenges
database system
springer
representation
different
modeling
digital
theory
multi
visualization
virtualization
issue
human
segmentation
real world
cse al
course course
course material
fundamental
function
reasoning
ac grae
csl department
cambridge university
grow hill
practical
modern
approach
sorting
time
understand
synthesis
gpu
nptel course
component
computational
evolution
tool
hardware
prerequisite data
compulsory prerequisite
pattern recognition
service modeling
www.international
kernel
course csl
engineering offered
xx department
development
journal
laboratory
number cs
link
log
image
visual
knowledge
video
program
advanced
cryptography
environment
related
set
xx communication
communication
information
method
structures algorithm
textbook
memory
objectives instructor
university press
data structure
engineering offered
computer vision
operating system
research
computer network
expected ability
elective prerequisite
programming
self learning
santanu chaudhury
concept
ph type
tech ph
social network
social
segmentation
real world
continuity
cse department
ai
lab
component
time
understand
synthesis
grow hill
practical
cambridge university
modern
approach
sorting
gpu
nptel course
computational
evolution
tool
hardware
prerequisite data
compulsory prerequisite
pattern recognition
service modeling
www.international
kernel
course csl
software
based
engineering structure
computer graphics
kishore kothapalli
search
work
challenges
database system
springer
representation
different
modeling
digital
theory
multi
visualization
virtualization
issue
human
segmentation
real world
cse al
course course
course material
fundamental
function
reasoning
ac grae
csl department
cambridge university
grow hill
practical
modern
approach
sorting
time
understand
synthesis
gpu
nptel course
component

Q1) Task 2)

Output Of Code

```
Total sentences: 1

==============================
Experiment 1 {'dim': 50, 'window': 3, 'negative': 5}
==============================

Training CBOW...

CBOW Evaluation:
Vocabulary size: 2508

Top 5 words similar to 'research':
techniques : 0.9987
system : 0.9986
problems : 0.9985
learning : 0.9985
design : 0.9985

Top 5 words similar to 'student':
research : 0.9888
space : 0.9885
laboratory : 0.9884
evaluation : 0.9877
network : 0.9877

Top 5 words similar to 'learning':
techniques : 0.9989
machine : 0.9988
models : 0.9987
research : 0.9985
```

```
research : 0.9985
design : 0.9985

Training Skip-gram...

Skip-gram Evaluation:
Vocabulary size: 2508

Top 5 words similar to 'research':
projects : 0.9912
exams : 0.9911
demonstrate : 0.9898
etc : 0.9893
able : 0.9892

Top 5 words similar to 'student':
undergraduate : 0.9895
well : 0.9871
suffix : 0.9868
ug : 0.9865
ica : 0.9863

Top 5 words similar to 'learning':
deep : 0.9401
machine : 0.9341
understand : 0.8827
recognition : 0.8812
students : 0.8736
```

```
==============================
Experiment 2 {'dim': 100, 'window': 5, 'negative': 5}
==============================

Training CBOW...

CBOW Evaluation:
Vocabulary size: 2508

Top 5 words similar to 'research':
algorithm : 0.9998
programming : 0.9998
models : 0.9998
architecture : 0.9998
techniques : 0.9997

Top 5 words similar to 'student':
languages : 0.9984
book : 0.9983
based : 0.9983
models : 0.9983
machines : 0.9983

Top 5 words similar to 'learning':
optimization : 0.9996
techniques : 0.9996
deep : 0.9996
theory : 0.9996
models : 0.9996
```

```
Training Skip-gram...

Skip-gram Evaluation:
Vocabulary size: 2508

Top 5 words similar to 'research':
student : 0.9718
academic : 0.9607
institute : 0.9603
would : 0.9539
assignment : 0.9531

Top 5 words similar to 'student':
undergraduate : 0.9765
postgraduate : 0.9749
ug : 0.9747
research : 0.9718
program : 0.9704

Top 5 words similar to 'learning':
deep : 0.9603
machine : 0.9579
pattern : 0.9242
recognition : 0.9182
human : 0.8583

==============================
Experiment 3 {'dim': 200, 'window': 8, 'negative': 10}
==============================
```

```
Training CBOW...

CBOW Evaluation:
Vocabulary size: 2508

Top 5 words similar to 'research':
knowledge : 0.9999
etc : 0.9999
projects : 0.9999
evaluation : 0.9999
study : 0.9999

Top 5 words similar to 'student':
program : 0.9998
programs : 0.9998
academic : 0.9998
work : 0.9997
surfaces : 0.9997

Top 5 words similar to 'learning':
deep : 0.9997
fundamentals : 0.9996
areas : 0.9996
theory : 0.9996
research : 0.9996

Training Skip-gram...
```

```
Training Skip-gram...

Skip-gram Evaluation:
Vocabulary size: 2508

Top 5 words similar to 'research':
student : 0.9006
institute : 0.9004
academic : 0.8932
work : 0.8822
students : 0.8764

Top 5 words similar to 'student':
undergraduate : 0.9395
postgraduate : 0.9297
ug : 0.9136
program : 0.9116
btech : 0.9086

Top 5 words similar to 'learning':
pattern : 0.935
machine : 0.9335
deep : 0.9212
recognition : 0.9181
beyond : 0.9084

Training completed successfully!
```

Q1.) Task 3.)

Output Of Code

```
========== CBOW Nearest Neighbors ==========

Top 5 neighbors for 'research':
algorithm : 0.9998
programming : 0.9998
models : 0.9998
architecture : 0.9998
techniques : 0.9997

Top 5 neighbors for 'student':
languages : 0.9984
book : 0.9983
based : 0.9983
models : 0.9983
machines : 0.9983

Top 5 neighbors for 'phd':
text : 0.9978
based : 0.9978
problem : 0.9978
computational : 0.9978
theory : 0.9978

Top 5 neighbors for 'exam':
machine : 0.9969
evaluation : 0.9969
architecture : 0.9968
different : 0.9968
models : 0.9968
```

riya_v21 copy.ipynb | segformer.ipynb | results.txt | biconnected.cu | q1.ipynb | q2.ipynb | TrainingNames.txt

nlu2 > q1.ipynb > from gensim.models import Word2Vec

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | heysk (Python 3.7

```
Top 5 neighbors for 'exam':
machine : 0.9969
evaluation : 0.9969
architecture : 0.9968
different : 0.9968
models : 0.9968

========== CBOW Analogy Tests ==========

ug : btech :: pg : ?
Answer: program ( 0.9959 )

phd : student :: research : ?
Answer: systems ( 0.9965 )

exam : course :: assignment : ?
Answer: courses ( 0.9956 )

========== Skip-gram Nearest Neighbors ==========

Top 5 neighbors for 'research':
student : 0.9718
academic : 0.9607
institute : 0.9603
would : 0.9539
assignment : 0.9531

Top 5 neighbors for 'student':
undergraduate : 0.9765
```

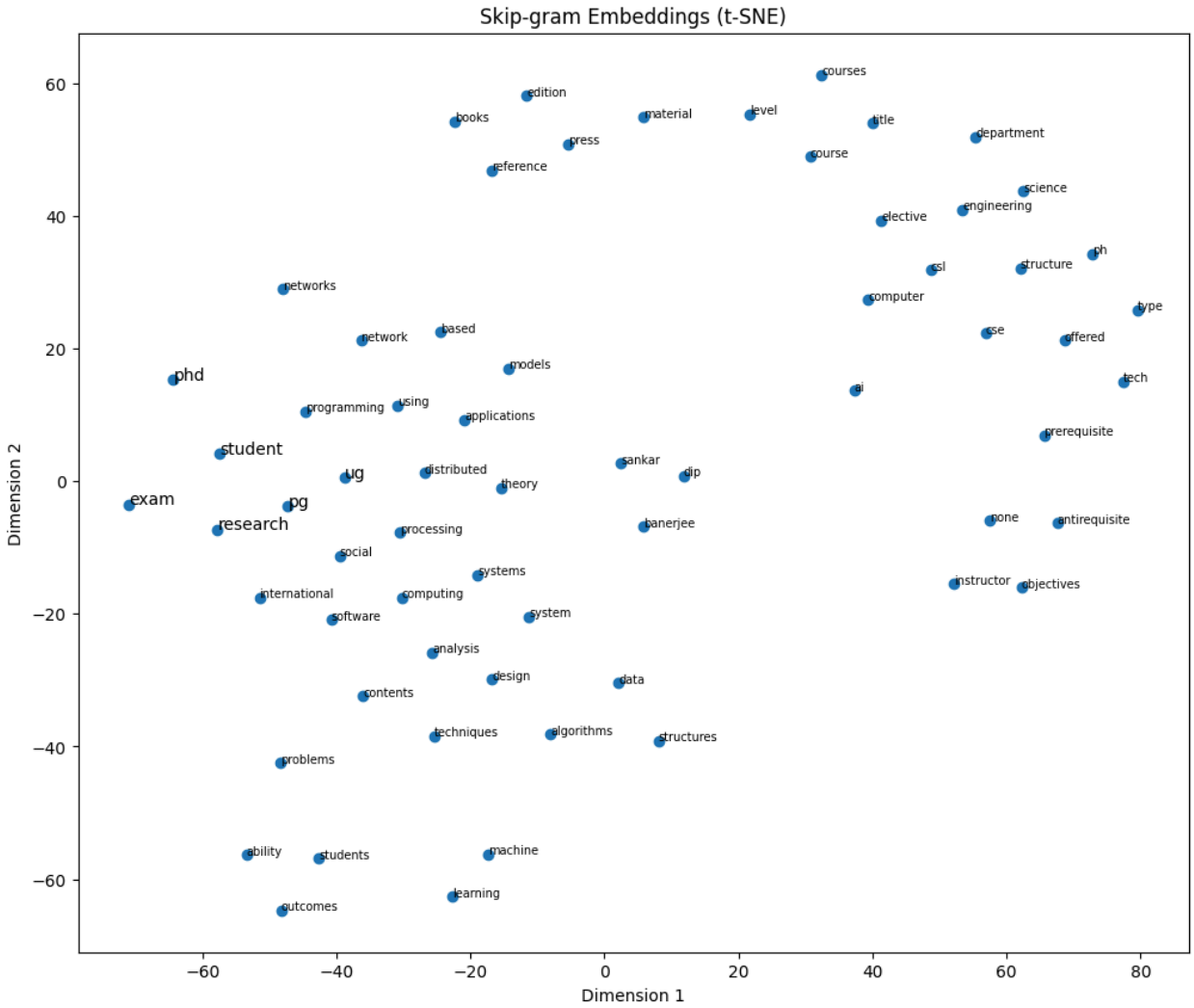
```
Top 5 neighbors for 'student':
undergraduate : 0.9765
postgraduate : 0.9749
ug : 0.9747
research : 0.9718
program : 0.9704

Top 5 neighbors for 'phd':
student : 0.9249
postgraduate : 0.9041
program : 0.9024
undergraduate : 0.9021
ug : 0.8994

Top 5 neighbors for 'exam':
assignment : 0.924
research : 0.9085
student : 0.8988
vision : 0.8795
institute : 0.8784

Top 5 neighbors for 'exam':
assignment : 0.924
research : 0.9085
student : 0.8988
vision : 0.8795
institute : 0.8784

========== Skip-gram Analogy Tests ==========
```

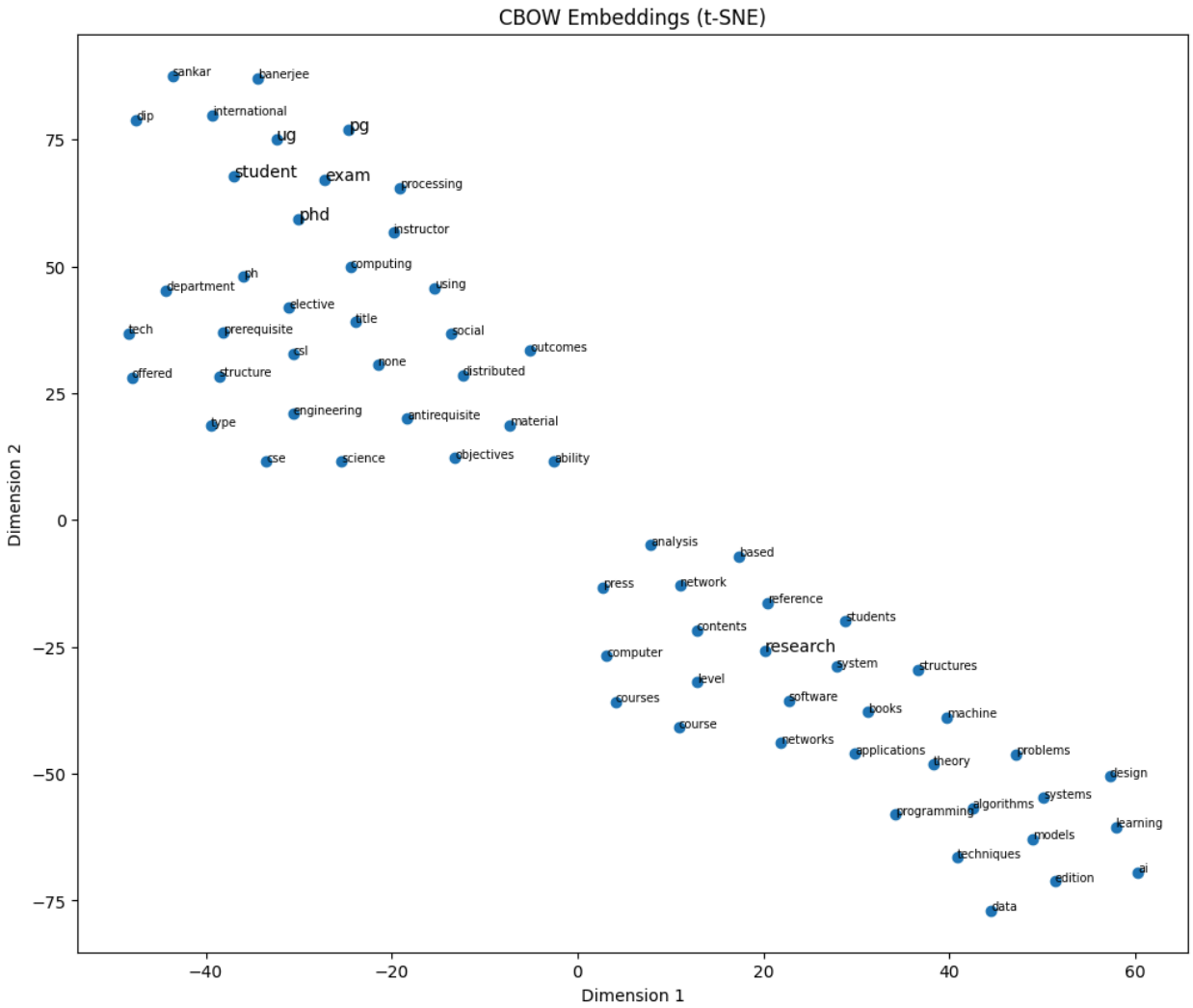
```
========== Skip-gram Analogy Tests ==========

ug : btech :: pg : ?
Answer: mtech ( 0.9823 )

phd : student :: research : ?
Answer: assignment ( 0.9271 )

exam : course :: assignment : ?
Answer: stanford ( 0.9511 )
```

Q1) Task 4)

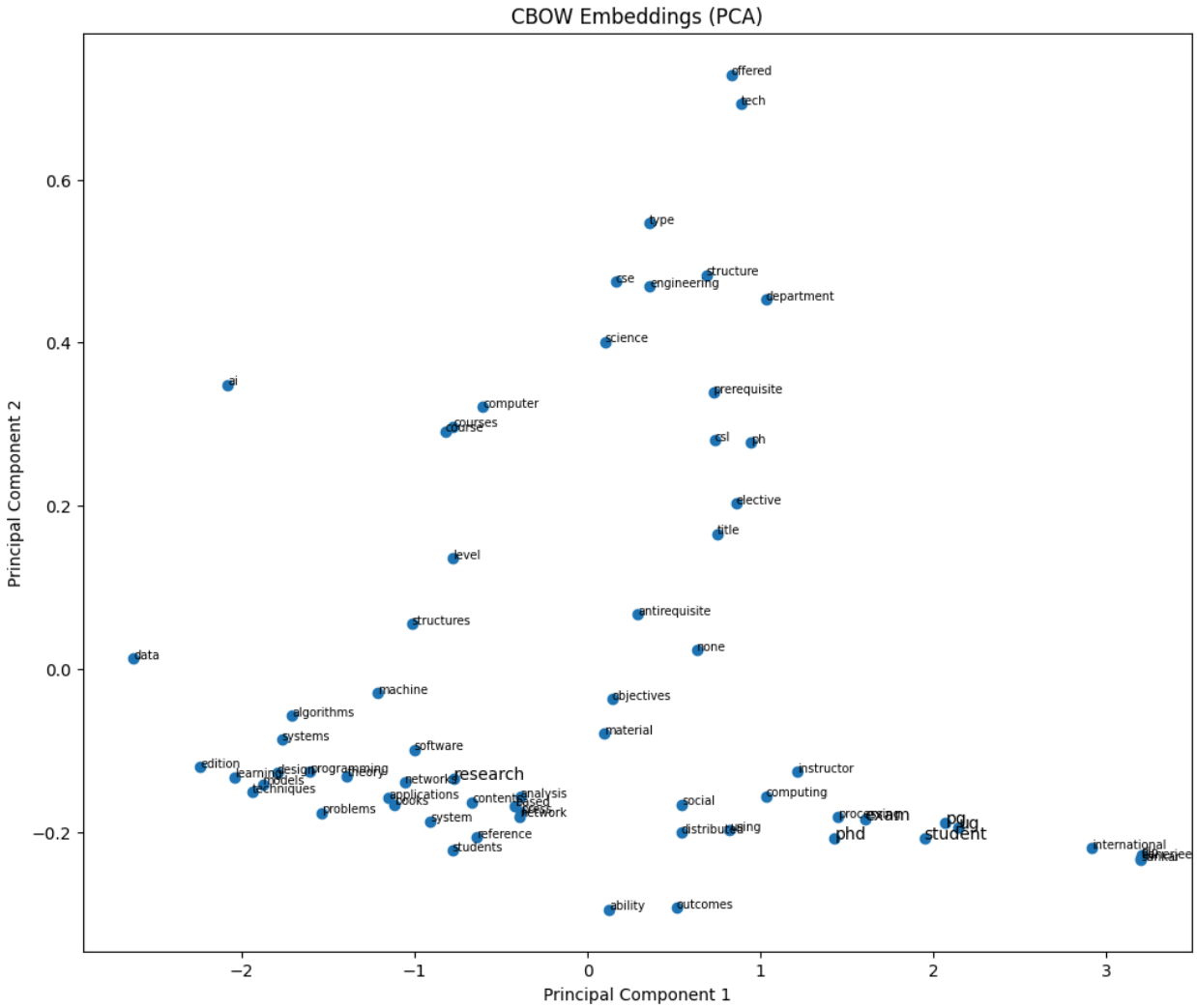
Output Of Code

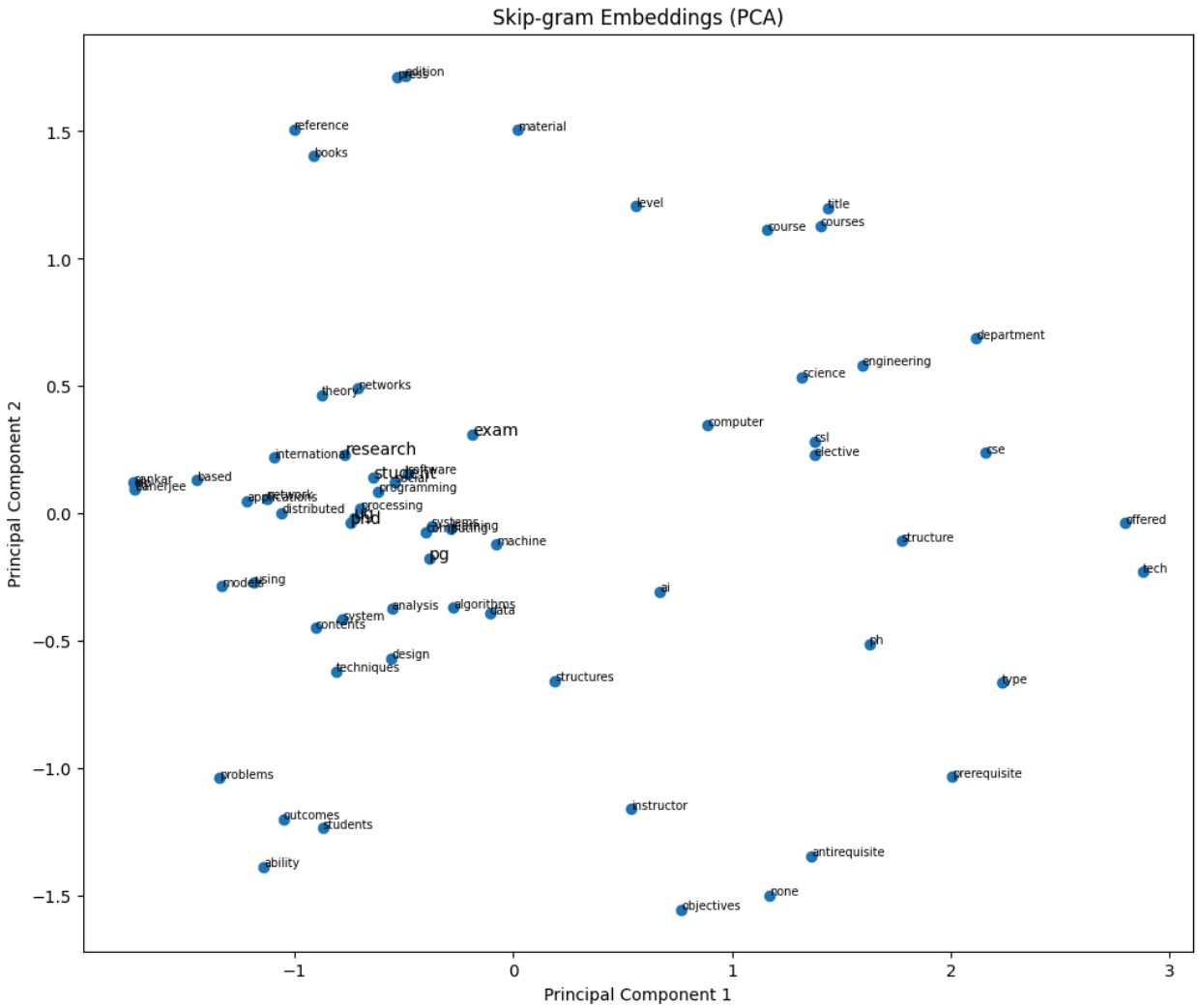
A.) t-SNE Plots


Skip-gram Embeddings (t-SNE)
Dimension 1
Dimension 2
courses
edition
books
material
level
title
department
press
course
reference
science
engineering
elective
ph
csl
structure
networks
computer
type
based
network
cse
offered
models
phd
tech
ai
programming
using
applications
prerequisite
student
sankar
ug
distributed
slp
theory
exam
pg
research
none
antirequisite
processing
banerjee
social
systems
instructor
objectives
international
computing
software
system
analysis
design
data
contents
techniques
algorithms
structures
problems
ability
students
machine
learning
outcomes


CBOW Embeddings (t-SNE)
Dimension 1
Dimension 2
sankar
banerjee
dip
international
ug
pg
student
exam
processing
phd
instructor
computing
department
ph
using
elective
title
tech
prerequisite
social
ssl
outcomes
none
offered
structure
distributed
engineering
antirequisite
material
type
cse
science
objectives
ability
analysis
based
press
network
reference
students
contents
computer
research
system
structures
level
courses
software
books
machine
course
networks
applications
theory
problems
design
systems
programming
algorithms
learning
models
techniques
ai
edition
data

A.) PCA Plots


CBOW Embeddings (PCA)
Principal Component 2
Principal Component 1
0.6
0.4
0.2
0.0
−0.2
−2
−1
0
1
2
3
offered
tech
type
structure
cse
engineering
department
science
ai
prerequisite
computer
courses
course
csl
ph
elective
title
level
entirequisite
structures
none
data
machine
objectives
algorithms
material
systems
software
edition
learning
models
techniques
research
network
instructor
computing
applications
analysis
social
problems
system
reference
students
phd
student
international
ability
outcomes



INTERPRETATION

Interpretation of Clustering Behavior

(A) t-SNE Visualization

The t-SNE plots show how words are grouped based on their semantic similarity in the embedding space.

In the **Skip-gram model**, clear and meaningful clusters are observed:

- Words like *“student”, “ug”, “pg”, “phd”* are grouped together, showing that the model captures academic hierarchy.

- Words such as *“exam”, “assignment”, “course”* appear close to each other, indicating evaluation-related concepts.
- *“research”* and *“phd”* are positioned near each other, reflecting a strong semantic relationship.

In contrast, the **CBOW model** shows:

- More scattered and less organized clusters
- Weaker grouping of related academic terms
- Less clear separation between different semantic categories

This suggests that **Skip-gram learns more meaningful and detailed relationships**, while **CBOW produces more general embeddings**.

(B) PCA Visualization

The PCA plots show less clear clustering compared to t-SNE.

- Words are more spread out and arranged in a linear manner
- Clusters are not sharply separated
- Some grouping of related words is still visible

This behavior is expected because:

- **PCA is a linear method**, so it cannot capture complex relationships
- **t-SNE is non-linear**, which makes clusters more visible and well-separated

(C) CBOW vs Skip-gram (Key Differences)

Aspect	CBOW	Skip-gram
Learning approach	Predicts word from context	Predicts context from word
Speed	Faster	Slower
Rare word handling	Weak	Strong
Semantic quality	General	Rich and precise
Clustering (observed)	Scattered	Well-formed clusters

From the visualizations, it is clear that the **Skip-gram model produces better embeddings**. It captures semantic relationships more effectively, which is reflected in the tight and meaningful clusters in the t-SNE plots.

On the other hand, **CBOW is computationally faster but produces more dispersed embeddings**, resulting in weaker semantic grouping.

Q2.) Task 1)

A.) Vanilla RNN

(i) Architecture

Vanilla RNN is a simple recurrent model used for character-level name generation. The model processes the input sequence one character at a time.

Each character is converted into a vector of size **64 (embedding size)**. At each time step, the hidden state is updated using the current input and previous hidden state:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

The hidden state is then passed through a linear layer to predict the next character. The model uses a **single recurrent layer** and does not have any gating mechanism.

(ii) Number of Trainable Parameters

Given:

- Vocabulary size (V) = 5868
- Embedding size (E) = 64
- Hidden size (H) = 64

Embedding:
5868×64=3755525868×64=375552

Input-to-hidden:
64×64=409664×64=4096

Hidden-to-hidden:
64×64=409664×64=4096

Bias:
6464

Output layer:
64×5868=37555264×5868=375552

Output bias:
58685868

Total Parameters

375552+4096+4096+64+375552+5868=765,228375552+4096+4096+64+375552+5868=765,228

Total ≈ 7.65 lakh parameters

(iii) Hyperparameters

- Embedding size: 64
- Hidden size: 64
- Number of layers: 1
- Learning rate: 0.0005
- Optimizer: Adam
- Epochs: 10

(iv) Observation

Vanilla RNN is simple and faster to train but cannot capture long-term dependencies effectively. This may reduce the quality of generated names compared to BLSTM and Attention models.

B.) BLSTM (Bidirectional LSTM)

(i) Architecture

BLSTM is an advanced recurrent model that uses LSTM cells and processes the sequence in both directions.

Each character is converted into a vector of size **64 (embedding size)**. Two LSTM layers are used:

- Forward LSTM (left → right)
- Backward LSTM (right → left)

Each LSTM uses input, forget, and output gates along with a cell state to capture long-term dependencies.

The outputs from both directions are concatenated to form a vector of size **128**. A dropout layer is applied, and then a linear layer predicts the next character.

(ii) Number of Trainable Parameters

Given:

- Vocabulary size (V) = 5868
- Embedding size (E) = 64
- Hidden size (H) = 64

LSTM parameters:

For one LSTM:

- Input weights: 64×256=1638464 *256 = 1638464×256=16384
- Hidden weights: 64×256=1638464 *256 = 1638464×256=16384
- Bias: 256256256

Total = **33024**

For two LSTMs:

2×33024=660482 * 33024 = 660482×33024=66048

Embedding:

5868×64=3755525868 * 64 = 3755525868×64=375552

Output layer:

128×5868+5868=756972128* 5868 + 5868 = 756972128×5868+5868=756972

Total Parameters

66048+375552+756972=1,198,57266048 + 375552 + 756972 = 1,198,57266048+375552+756972=1,198,572

Total ≈ 1.2 million parameters

(iii) Hyperparameters

- Embedding size: 64
- Hidden size: 64
- Number of layers: 1 (per direction)
- Learning rate: 0.0005
- Optimizer: Adam
- Epochs: 10
- Dropout: 0.3

(iv) Observation

BLSTM gives better results than Vanilla RNN because it uses both past and future context. It also handles long-term dependencies better due to LSTM cells. However, it has a large number of parameters and is slower to train.

C.) Attention RNN

(i) Architecture

Attention RNN extends the Vanilla RNN by adding an attention mechanism. The model first processes the input sequence using a basic RNN to generate hidden states for all time steps.

Each character is converted into a vector of size **64 (embedding size)**. A hidden state of size **64** is computed at each step using the RNN.

After that, attention is applied:

- The current hidden state acts as a **query**
- All past hidden states act as **keys and values**
- The model computes attention scores to decide which past positions are important

A **causal mask** is used so the model does not look at future characters.

A context vector is computed as a weighted sum of past states. This context is combined with the current hidden state and passed through a linear layer to predict the next character.

(ii) Number of Trainable Parameters

Given:

- Vocabulary size (V) = 5868
- Embedding size (E) = 64
- Hidden size (H) = 64

Embedding:

5868×64=3755525868 * 64 = 3755525868×64=375552

RNN parameters:

- Input-to-hidden: 64×64=409664 * 64 = 409664×64=4096
- Hidden-to-hidden: 64×64=409664 * 64 = 409664×64=4096
- Bias: 646464

Total = **8256**

Attention (Q, K, V layers):

Each has:

64×64+64=416064 *64 + 64 = 416064×64+64=4160

Total for 3 layers:

3×4160=124803 *4160 = 124803×4160=12480

Output layer:

128×5868+5868=756972128 *5868 + 5868 = 756972128×5868+5868=756972

Total Parameters

375552+8256+12480+756972=1,153,260375552 + 8256 + 12480 + 756972 = 1,153,260375552+8256+12480+756972=1,153,260

Total ≈ 11.5 lakh parameters

(iii) Hyperparameters

- Embedding size: 64
- Hidden size: 64
- Number of layers: 1
- Learning rate: 0.0003
- Optimizer: Adam
- Epochs: 15
- Dropout: 0.3

(iv) Observation

Attention RNN performs better than both Vanilla RNN and BLSTM because it can focus on important parts of the sequence while making predictions. This improves the quality and diversity of generated names. However, it is more computationally expensive.

Q2. Task 2.)

Output Of Code

riya_v21 copy.ipynb | segformer.ipynb | results.txt | biconnected.cu | q1.ipynb | q2.ipynb | TrainingNames.txt

lu2 > q2.ipynb > import torch

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | heysk (Python 3.7.3)

```python
gen = generate_samples(model)

print("Novelty:", compute_novelty(gen, names))
print("Diversity:", compute_diversity(gen))
print("Samples:", gen[:5])
```

Python

```
Training Vanilla RNN...
Epoch 1, Loss: 34532.82
Epoch 2, Loss: 26996.07
Epoch 3, Loss: 24415.59
Epoch 4, Loss: 22740.41
Epoch 5, Loss: 21618.13
Epoch 6, Loss: 20730.94
Epoch 7, Loss: 20051.72
Epoch 8, Loss: 19520.00
Epoch 9, Loss: 18987.90
Epoch 10, Loss: 18570.72
Training BLSTM...
Epoch 1, Loss: 19301.37
Epoch 2, Loss: 3593.41
Epoch 3, Loss: 1336.27
Epoch 4, Loss: 671.00
Epoch 5, Loss: 382.65
Epoch 6, Loss: 217.78
Epoch 7, Loss: 151.65
Epoch 8, Loss: 101.28
Epoch 9, Loss: 73.52
Epoch 10, Loss: 52.80

Training Attention...
Epoch 1, Loss: 37606.86
Epoch 2, Loss: 30542.13
Epoch 3, Loss: 27779.07
Epoch 4, Loss: 26265.04
```


```
Epoch 1, Loss: 37606.86
Epoch 2, Loss: 30542.13
Epoch 3, Loss: 27779.07
Epoch 4, Loss: 26265.04
```


```
Epoch 5, Loss: 25080.90
Epoch 6, Loss: 24321.92
Epoch 7, Loss: 23529.78
Epoch 8, Loss: 22846.84
Epoch 9, Loss: 22256.49
Epoch 10, Loss: 21761.89
Epoch 11, Loss: 21296.13
Epoch 12, Loss: 21019.58
Epoch 13, Loss: 20607.86
Epoch 14, Loss: 20268.13
Epoch 15, Loss: 19993.47

=== VanillaRNN ===
Novelty: 1.0
Diversity: 1.0
Samples: ['kizaa mulrad nair', 'sheetal redwa', 'batesh kumar', 'meeya ali', 'rizala naun gupta']

=== BLSTM ===
Novelty: 1.0
Diversity: 0.96
Samples: ['usiv', 'anum', 'wamy', 'fzuqkun', 'izz']

=== Attention ===
Novelty: 0.995
Diversity: 1.0
Samples: ['raduvatu barju nair', 'fosh zail', 'naka mehta', 'jakhas imehti iyar', 'barsnchmiran boor']
```

Q2.) Task 3

Qualitative Analysis

1. Vanilla RNN

Realism:
The Vanilla RNN generates names that look somewhat realistic but are often inconsistent. Some names resemble real Indian names (e.g., *"sheetal redwa", "batesh kumar"*), but many contain unusual combinations or extra words. The model can learn basic patterns like common surnames but fails to maintain consistency across the full name.

Common Failure Modes:

- Produces extra words or long names unnecessarily
- Weak handling of long sequences
- Repetition or unnatural character combinations
- Limited ability to capture full name structure

Sample Outputs:

- kizaa mulrad nair
- sheetal redwa
- batesh kumar
- meeya ali
- rizala naun gupta

2. BLSTM

Realism:
BLSTM generates shorter and more structured outputs compared to Vanilla RNN. However, many generated names are too short or incomplete (e.g., *"anum", "izz"*). While it captures character patterns better, it sometimes fails to generate full meaningful names.

Common Failure Modes:

- Very short or incomplete names
- Loss of full name structure (missing surname)

- Sometimes generates random character sequences

Sample Outputs:

- usiv
- anum
- wamy
- fzuqkun
- izz

3. Attention RNN

Realism:
Attention RNN produces more realistic and structured names compared to both Vanilla RNN and BLSTM. It is able to generate names with proper first and last name patterns (e.g., *"naka mehta"*). The attention mechanism helps the model focus on relevant parts of the sequence.

Common Failure Modes:

- Occasionally generates slightly distorted names
- Some names contain extra or incorrect characters
- Rare cases of unrealistic combinations

Sample Outputs:

- raduvatu barju nair
- fosh zail
- naka mehta
- jakhas imehti iyar
- barsnchmiran boor

Overall Comparison

- Vanilla RNN: Basic learning, less consistent
- BLSTM: Better sequence understanding but sometimes too short
- Attention RNN: Best performance with more realistic and structured names

---